PythonDiscreteControl — Documentation

Release 1.0

Adam Caulier

Table of Contents

A Python project for the analysis, design, and simulation of discrete-time control systems. The plant studied is the **ball-and-beam** modelled as a double integrator . Two control strategies are compared: a **discrete PID controller** and a **polynomial RST regulator**.

1 Overview

1.1 Architecture

The project is organised in independent layers:

```
Notebooks
  |-- Simulation/runners.py          <- high-level wiring (D=2, sat=+/-10, t=3s)
       |-- Simulation/simulation.py  <- TFSimulator, HybridSim, NonLinearHybridSim
       |-- Control/                  <- DiscretePID, RSTController
       |-- Models/BallBeam/          <- config, TransferFunctionModel, StateSpace, Nonlinear
       `-- Metrics_Plotting/         <- SimLog, Metrics, Plotting
Utils/computeRST.py                  <- RST synthesis (no circular dependencies)
Utils/utils.py                       <- CSV export, utility builders
```

1.2 Plant model

The **ball-and-beam** system is described by the continuous transfer function:

(1.1)

After ZOH discretisation at s:

(1.2)

The two poles at (discrete double integrator) make the system non-asymptotically stable in open loop.

1.3 Common controller interface

Both `DiscretePID` and `RSTController` share three methods:

- `reset()` — clears the internal state to zero before each simulation run.
- `setReference(r)` — sets the reference setpoint.
- `step(y) -> float` — computes the control command from measurement .

This common interface allows one controller to be substituted for the other in the runner functions without changing any simulation code.

1.4 Hardcoded constants in the runners

Constant	Value	Role
Disturbance (ampl.)	2.0	Step disturbance injected at the plant input
Disturbance (time)	t >= 3s	First 3 s reserved for the reference-tracking transient
Saturation	+/-10	Physical limit of the beam servo actuator
Integration step	1 ms	20x finer than the controller period (50 Hz)

1.5 RST synthesis

Two synthesis functions are available in `Utils/computeRST.py`:

`Compute_Denominator_Matching_RST(A_m, plant_tf, Integrator, A0)`
Specify only the dominant poles . The polynomial enforces unity DC gain and cancels from the closed-loop response.

`Compute_Desired_RST(Desired_TF, plant_tf, Integrator, A0)`
Specify the full desired closed-loop transfer function . The numerator must be exactly divisible by .

Both functions return `(S_tf, R_tf, T_tf, H_cl)`.

2 API Reference

2.1 Models

Abstract base classes for plant models.

Any new plant must implement these interfaces so that the Simulation layer remains indifferent to the specific plant implementation.

class Models.base.**BaseNonlinearModel**
Bases: ABC
Abstract base for nonlinear plant models.
Subclasses must: - Set `self.C` in `__init__`. - Implement `f(X, u)` returning the state derivative $\dot{X}$.

C: ndarray

abstractmethod f (*X: ndarray*, *u: float*) → ndarray
State derivative $\dot{X} = f(X, u)$.

Parameters
- **X** – State vector, shape (n, 1).
- **u** – Scalar control input.

Returns State derivative, same shape as X.

class Models.base.**BaseLinearStateSpaceModel**

Bases: object

Base class for linearised continuous/discrete state-space plant models.

Subclasses must populate the following attributes in __init__:

Continuous:

A, B, C, D — system matrices (numpy arrays)

Discrete (ZOH):

Ad, Bd, Cd, Dd — discretised system matrices (numpy arrays)

A: ndarray

B: ndarray

C: ndarray

D: ndarray

Ad: ndarray

Bd: ndarray

Cd: ndarray

Dd: ndarray

2.1.1 BallBeam — configuration

Ball-and-beam system physical parameters and simulation settings.

All values are taken from the CTMS reference model: https://ctms.engin.umich.edu/CTMS/index.php?example=BallBeam&section=ControlStateSpace

Physical constants

m : *float*

Ball mass [kg].

R : *float*

Ball radius [m].

g : *float*

Gravitational acceleration [m/s²].

J : *float*

Ball moment of inertia.

d : *float*

Lever arm from the beam pivot to the servo attachment point [m].

L : *float*

Total beam length [m].

H : *float*

Linearised plant gain.

Simulation settings

dt : *float*

Controller sampling period [s]. dt = 1/50 = 0.02 s => 50 Hz sampling rate.

T : *float*

Total simulation duration [s].

2.1.2 BallBeam — transfer functions

class Models.BallBeam.TransferFunctions.**TransferFunctionModel** (*config_file*)

Bases: object

Continuous and ZOH-discretised transfer function for a generic plant.

Tf_cont
Continuous-time transfer function built from config.

Type ct.TransferFunction

Tf_dis
ZOH-discretised transfer function at period `config_file.dt`.

Type ct.TransferFunction

num_dis
Numerator coefficients of the discrete TF, shape (nb,).

Type np.ndarray

den_dis
Denominator coefficients of the discrete TF, shape (na+1,).

Type np.ndarray

The config must expose:
`num_cont` — list of continuous numerator coefficients `den_cont` — list of continuous denominator coefficients `dt` — sampling period [s]

2.1.3 BallBeam — linear state space

class Models.BallBeam.StateSpace.**LinearStateSpaceModel** (*config_file*)
Bases: BaseLinearStateSpaceModel
Linearised continuous and ZOH-discrete state-space model for a generic plant.
Populates both the continuous attributes (A, B, C, D) defined by `BaseLinearStateSpaceModel` and the discrete attributes (Ad, Bd, Cd, Dd) obtained by ZOH sampling at `config_file.dt`.

The config must expose:
`A_mat`, `B_mat`, `C_mat`, `D_mat` — continuous system matrices `dt` — sampling period [s]

2.1.4 BallBeam — nonlinear dynamics

Nonlinear dynamics of the ball-and-beam system.

The state is X = [[r], [ṙ]] where r is the ball position [m] and ṙ its velocity [m/s]. The control input u is the servo beam angle [rad]. The nonlinear ODE is:

$$\dot{r} = \dot{r}\ \ddot{r} = -m \cdot g \cdot \sin(\alpha) \ /\ (J/R^2 + m), \quad \alpha = (d/L) \cdot u$$

class Models.BallBeam.NonlinearDynamics.**NonlinearBallBeamModel** (*config_file*)
Bases: BaseNonlinearModel
Nonlinear model of the ball-and-beam plant.
Implements the exact (non-linearised) equations of motion using sin(α) instead of the small-angle approximation sin(α) ≈ α used by the linear models. For small inputs the nonlinear and linear responses are nearly identical; differences become visible when `abs(u) > ~0.1 rad`.

f (*X: ndarray*, *u: float*) → ndarray
Evaluates the nonlinear state derivative Ẋ = f(X, u).

Parameters
- **X** (*np.ndarray*) – State vector [[r], [ṙ]], shape (2, 1). r — ball position along the beam [m]. ṙ — ball velocity [m/s].
- **u** (*float*) – Servo beam angle [rad].

Returns State derivative [[ṙ], [r̈]], shape (2, 1).

Return type np.ndarray

2.2 Control

Discrete PID controller (backward Euler) with optional derivative filter.

The PID is expressed as a discrete transfer function C(z) = R(z)/S(z) and also exposes R, S, T polynomials so it can be used wherever an `RSTController` is expected (unity-feedback: T = R, S = denominator of C).

class Control.DiscretePID.**DiscretePID** (*kp*, *ki*, *kd*, *dt*, *text_option: str = 'NotFiltered'*)

Bases: `object`

Discrete PID controller discretised with backward Euler.

Supports filtered and unfiltered derivative modes. Does not implement output saturation (saturation is handled by the runner functions). Also exposes the equivalent RST polynomial representation for interoperability with `RSTController`.

reset ( )

Resets the controller's internal state to zero.

Call this before each new simulation run to clear accumulated integral and derivative history from the previous run.

setReference (*r*)

Sets the reference setpoint for the controller.

Parameters **r** (*float*) – Desired plant output (setpoint).

step (*y*)

Computes the control output u[k] for the current measurement y[k].

Computes the error e[k] = r – y[k] and passes it through the PID transfer function implemented as a difference equation in `TFSimulator`.

Parameters **y** (*float*) – Current plant output (measured value).

Returns Control signal u[k].

Return type float

As_TransferFunction (*text_option: str*)

Builds the discrete PID transfer function (position form, backward Euler).

Both modes share the same P and I terms (backward Euler, s => (z–1)/(dt·z)):

P(z) = Kp I(z) = Ki·dt·z / (z – 1)

"filtered"

Derivative with first-order low-pass filter (N = 50): D(z) = Kd·N·(z – 1) / ((N·dt + 1)·z – 1) Filter pole at z = 1/(1 + N·dt) — always inside the unit circle.

Any other value (default "NotFiltered")

Pure backward-Euler derivative (no filter): D(z) = Kd·(z – 1) / (dt·z)

Parameters **text_option** (*str*) – `"filtered"` or anything else.

Returns Discrete PID transfer function.

Return type ct.TransferFunction

As_RST ( )

Converts the PID transfer function into equivalent R, S, T polynomials.

For a unity-feedback PID the RST representation is:

```
R(z) = T(z) = numerator of C(z)
S(z) = denominator of C(z)
```

This lets a PID be used anywhere an `RSTController` is expected.

Polynomial RST controller (two-degree-of-freedom).

The RST polynomials are typically produced by one of the synthesis functions in `Utils/computeRST.py` and then passed to `RSTController.__init__`. A `DiscretePID` instance is also accepted anywhere an `RSTController` is

expected because both expose the same `reset` / `setReference` / `step` interface.

class Control.RSTController.**RSTController** (*R: TransferFunction*, *S: TransferFunction*, *T: TransferFunction*)

Bases: object

Polynomial RST controller implementing the two-degree-of-freedom control law:

$$S(z) \cdot u[k] = T(z) \cdot r[k] - R(z) \cdot y[k]$$

Equivalently:

v[k] = T·r_hist − R·y_hist (FIR dot products on reference and output histories) u[k] = (1/S) · v[k] (recursive IIR filter realised by TFSimulator)

The two degrees of freedom mean that T and R can be tuned independently: T shapes the reference tracking response while R shapes disturbance rejection without affecting the other.

reset ( )

Resets the controller's internal state to zero.

Clears the reference history, output history, and the 1/S recursive filter state. Call this before each new simulation run to avoid carry-over from a previous run.

setReference (*r*)

Sets the reference setpoint for the controller.

Parameters **r** (*float*) – Desired plant output (setpoint).

step (*y*)

Computes the control output u[k] for the current plant measurement y[k].

Implements one step of the RST control law:

v[k] = T·r_hist − R·y_hist u[k] = (1/S) · v[k]

Parameters **y** (*float*) – Current plant output y[k].

Returns Control signal u[k].

Return type float

2.3 Simulation

Low-level discrete and hybrid plant simulators.

Three simulators are provided:

- `TFSimulator` — discrete transfer function stepped via a difference equation.
- **`HybridSim` — continuous linear plant (state-space) driven by a discrete controller,** integrated with RK4 at 1 ms.
- `NonLinearHybridSim`— same structure but with a nonlinear ODE from `BaseNonlinearModel`.

Runner functions that wire simulators to controllers live in `Simulation/runners.py`.

class Simulation.simulation.**TFSimulator** (*tf*, *X_0*)

Bases: object

Simulates a discrete transfer function step-by-step using its difference equation.

Given a transfer function B(z)/A(z), each call to `step` advances the recursion:

```
a0·y[k] = b0·u[k] + b1·u[k-1] + … − a1·y[k-1] − a2·y[k-2] − …
```

Used for both plant simulation (open-loop TF) and internally by `DiscretePID` and `RSTController` for their own difference-equation recursion.

reset (*X_0*)

Resets the input and output history buffers to zero.

Call this before starting a new simulation run to avoid carry-over from a previous run.

Parameters **X_0** (*float*) – Unused; kept for interface consistency with the runner functions.

step (*u*)

Advances the difference equation by one sample and returns the new output y[k].

Implements:

y[k] = (b0·u[k] + b1·u[k-1] + … − a1·y[k-1] − …) / a0

Parameters **u** (*float*) – Current input sample u[k].

Returns Current output sample y[k].

Return type float

class Simulation.simulation.**HybridSim** (*StateSpaceModel: BaseLinearStateSpaceModel, config_file*)

Bases: object

Hybrid simulator: continuous linear plant (state-space) driven by a discrete controller.

The plant is integrated at a fine timestep dt_plant = 1 ms (1 kHz) using fourth-order Runge-Kutta (RK4), while the controller output is updated at the coarser config_file.dt (typically 20 ms / 50 Hz). This ratio gives 20 RK4 substeps per control period, which is enough to make the integration error negligible compared with sampling effects.

rk4_step (*X, u_k*)

Advances the continuous state one integration step using classical RK4.

Integrates $\dot{X} = A \cdot X + B \cdot u$ over one dt_plant interval with the control input u_k held constant (zero-order hold within the substep).

Parameters

- **X** (*np.ndarray*) – Current state vector X[k], shape (n, 1).
- **u_k** (*np.ndarray*) – Control input applied during this step, shape (m, 1).

Returns Updated state vector X[k+1], shape (n, 1).

Return type np.ndarray

class Simulation.simulation.**NonLinearHybridSim** (*model: BaseNonlinearModel, config_file*)

Bases: object

Hybrid simulator: generic nonlinear plant driven by a discrete controller.

Identical structure to HybridSim but uses the nonlinear ODE f(X, u) from a BaseNonlinearModel subclass instead of linear state-space matrices. The plant is integrated at dt_plant = 1 ms using RK4; the controller output is updated at the coarser config_file.dt. The output matrix C is read from the model.

rk4_step (*X, u_k*) → ndarray

Advances the nonlinear plant state one integration step using classical RK4.

Integrates $\dot{X} = f(X, u)$ over one dt_plant interval with the control input u_k held constant (zero-order hold within the substep).

Parameters

- **X** (*np.ndarray*) – Current state vector [[r], [ṙ]], shape (2, 1).
- **u_k** (*np.ndarray*) – Control input (servo command), shape (1, 1) or scalar.

Returns Updated state vector [[r], [ṙ]], shape (2, 1).

Return type np.ndarray

2.3.1 Runners

High-level simulation runner functions.

Each runner wires together a plant simulator, an optional controller, and a SimLog, then returns the populated logger. Knowing the signature and side-effects of each runner is the primary interface point for users of this library.

Hardcoded values shared by all runners

Disturbance magnitude : *2.0 (same units as the control input)*

A constant step disturbance of 2.0 is added to the plant input for all samples after t = 3 s. The plant receives u + d while the logged control signal is the raw controller output. Because RST enforces S(1) = 0 (integrator in S), the steady-state position error is exactly zero. A PID without integral action (Ki = 0) settles with a permanent

offset of approximately D * S(1) / R(1) ≈ D / Kp.

Disturbance onset : *t > 3.0 s*

The first 3 s are reserved for the reference-tracking transient. The disturbance is injected after this window so that tracking and rejection performance can be assessed independently from a single simulation run.

Saturation limits : *±10 (same units as the control input)*

The control signal is clipped to [−10, +10]. These limits approximate the physical saturation of the ball-and-beam servo actuator (roughly ±10° of beam tilt in the normalised model). Saturation prevents integrator wind-up from driving the ball off the beam during large transients.

`Simulation.runners.`**`run_discrete_control`** `(` *`plant_sim: TFSimulator, controller, config_file, r: float, y_0: float, logger: SimLog, Disturb=True, Saturate=True`* `) → SimLog`

Runs the discrete closed-loop simulation and returns the populated logger.

The plant is modelled as a discrete transfer function stepped via `TFSimulator`. The controller is sampled at every step (one call to `controller.step` per `config_file.dt` interval).

Hardcoded values

Disturbance magnitude : *2.0*

A step disturbance of 2.0 is added to the plant input `u + d` for all samples `k >= int(3.0 / dt)`. The logged control signal is the raw controller output. RST (S(1) = 0) returns the position to `r` exactly; PID (Ki = 0) settles with a permanent offset ≈ D / Kp. See the module docstring for the rationale.

Saturation limits : *[−10, +10]*

The raw controller output is clipped to this range before being sent to the plant. See the module docstring for the rationale.

param plant_sim	Simulator initialised with the plant's discrete transfer function.
type plant_sim	TFSimulator
param controller	Controller object exposing `step(y)` and `setReference(r)` methods.
param config_file	Plant configuration module; must expose `T` (total time [s]) and `dt` (sampling period [s]).
param r	Reference (setpoint) for the controller.
type r	float
param y_0	Initial plant output.
type y_0	float
param logger	SimLog instance used to record time, output, and input.
type logger	SimLog
param Disturb	If True, adds a step input disturbance of 2.0 to the plant input for all samples from t = 3 s onward. Defaults to True.
type Disturb	bool, optional
param Saturate	If True, clips the control signal to [−10, +10]. Defaults to True.
type Saturate	bool, optional
returns	The logger passed as input, now populated with simulation data.
rtype	SimLog

`Simulation.runners.`**`run_discrete_impulse_response`** `(` *`plant_sim: TFSimulator, config_file, y_0: float, logger: SimLog`* `) → SimLog`

Runs the open-loop discrete impulse response and returns the populated logger.

The impulse magnitude is 1/dt so that the discrete impulse approximates a continuous Dirac delta with unit area (∫u dt ≈ (1/dt)·dt = 1). After the first sample, u[k] = 0 for all remaining steps.

Hardcoded values

Impulse magnitude : *1/config_file.dt*

Chosen to normalise the impulse energy to 1, consistent with the continuous-domain convention. For dt = 0.02 s this gives u[0] = 50.

param plant_sim	Simulator initialised with the plant's discrete transfer function.
type plant_sim	TFSimulator

param config_file Plant configuration module; must expose `T` and `dt`.

param y_0 Initial plant output.

type y_0 float

param logger SimLog instance used to record time, output, and input.

type logger SimLog

returns The logger passed as input, now populated with simulation data.

rtype SimLog

`Simulation.runners.`**`run_discrete_step_response`** (*`plant_sim: TFSimulator`*, *`config_file`*, *`y_0: float`*, *`logger: SimLog`*) → `SimLog`

Runs the open-loop discrete step response and returns the populated logger.
A constant unit input u = 1.0 is applied for the full simulation duration.

Hardcoded values

Step magnitude : *1.0*
Unit step input applied at k = 0 and held constant. Scale the result by any other magnitude to obtain the step response for a different input level.

param plant_sim Simulator initialised with the plant's discrete transfer function.

type plant_sim TFSimulator

param config_file Plant configuration module; must expose `T` and `dt`.

param y_0 Initial plant output.

type y_0 float

param logger SimLog instance used to record time, output, and input.

type logger SimLog

returns The logger passed as input, now populated with simulation data.

rtype SimLog

`Simulation.runners.`**`run_continuous_control_loop`** (*`Simulator`*, *`controller`*, *`r`*, *`X_0`*, *`Logger: SimLog`*, *`Disturb=True`*, *`Saturate=True`*) → `SimLog`

Runs the closed-loop hybrid simulation (continuous plant, discrete controller).
The plant is integrated at `dt_plant` = 1 ms using RK4; the controller output is updated every `config_file.dt` seconds (zero-order hold between updates). Within each control period, `N_substep = config_file.dt / dt_plant` RK4 steps are taken. Every RK4 substep is logged, so the output log has a 1 ms resolution regardless of the controller sampling period.

Hardcoded values

Disturbance magnitude : *2.0*
A step disturbance of 2.0 is added to the controller output before it is sent to the plant for all control periods starting at t = 3 s. The plant receives `u + d` via `u_k`; the logged control signal is the raw controller output. RST (S(1) = 0) returns to `r` exactly; PID (Ki = 0) settles with a permanent offset ≈ D / Kp. See the module docstring for the rationale.

Saturation limits : *[−10, +10]*
The controller output is clipped to this range before the ZOH hold. See the module docstring for the rationale.

param Simulator Hybrid simulator initialised with the plant's state-space model. Must expose `C`, `config_file`, `dt_plant`, and `rk4_step`.

type Simulator HybridSim or NonLinearHybridSim

param controller Controller object exposing `step(y)` and `setReference(r)` methods.

param r Reference (setpoint) for the controller.

param X_0 Initial state vector of the continuous plant.

param Logger SimLog instance used to record time, output, and input.

type Logger SimLog

param Disturb If True, adds a step input disturbance of 2.0 to the plant input from t = 3 s onward. Defaults to True.

type Disturb bool, optional

param Saturate If True, clips the control signal to [−10, +10]. Defaults to True.

type Saturate bool, optional

returns The logger passed as input, now populated with simulation data.

rtype SimLog

`Simulation.runners.`**`run_continuous_impulse_response`** (*HybridSim*, *X_0*, *Logger:* *SimLog*) → SimLog

Runs the open-loop continuous impulse response using RK4 integration.
The impulse magnitude is 1/dt_plant so that the discrete pulse approximates a continuous Dirac delta with unit area, consistent with `run_discrete_impulse_response`. After the first integration step, u = 0 for all remaining steps.

Hardcoded values

Impulse magnitude : *1/HybridSim.dt_plant*
With dt_plant = 1e-3 s this gives u[0] = 1000, normalising the impulse energy to 1.

param HybridSim
Hybrid simulator initialised with the plant's state-space model.

type HybridSim HybridSim

param X_0 Initial state vector of the continuous plant.

param Logger SimLog instance used to record time, output, and input.

type Logger SimLog

returns The logger passed as input, now populated with simulation data.

rtype SimLog

`Simulation.runners.`**`run_continuous_step_response`** (*HybridSim*, *X_0*, *Logger:* *SimLog*) → SimLog

Runs the open-loop continuous step response using RK4 integration.
A constant unit input u = 1.0 is applied for the full simulation duration.

Hardcoded values

Step magnitude : *1.0*
Unit step input held constant from t = 0. Scale the result for other magnitudes.

param HybridSim
Hybrid simulator initialised with the plant's state-space model.

type HybridSim HybridSim

param X_0 Initial state vector of the continuous plant.

param Logger SimLog instance used to record time, output, and input.

type Logger SimLog

returns The logger passed as input, now populated with simulation data.

rtype SimLog

2.4 Utils

`Utils.computeRST.`**`trim`** (*p*)

Removes leading zeros from a polynomial coefficient array.
Leading zeros represent unnecessary high-order terms (e.g. [0, 0, 1] is just [1]). If the entire array is zero the function returns [0.0] rather than an empty array, so that downstream polynomial operations always receive a valid input.

Parameters **p** (*array-like*) – Polynomial coefficients in descending order of power.

Returns Trimmed coefficient array with no leading zeros (minimum length 1).

Return type np.ndarray

`Utils.computeRST.`**`conv_matrix`** `( a, n )`

Builds the convolution (Toeplitz) matrix for polynomial multiplication.
Returns a matrix M such that M @ x == np.convolve(a, x) for any length-n vector x. This lets the Diophantine equation A·S + B·R = A_m be written as a linear system M·θ = A_m and solved with least squares.
The returned matrix has shape (len(a) + n − 1, n).

Parameters
- **a** (*array-like*) – Polynomial coefficient array (the fixed polynomial).
- **n** (*int*) – Length of the unknown polynomial x (number of columns).

Returns Toeplitz convolution matrix of shape (len(a) + n − 1, n).

Return type np.ndarray

`Utils.computeRST.`**`Compute_Denominator_Matching_RST`** `( A_cl: list, plant_discrete_tf, Integrator=True, A0=None )`

RST synthesis by denominator (characteristic polynomial) matching.
Computes S, R, T by solving the Diophantine equation and setting T = t0 · A0. The A0 factor in T cancels with the A0 factor in the closed-loop denominator:

H_ry = B·T / (A·S + B·R) = B·(t0·A0) / (A_m·A0) = t0·B / A_m

so the reference-to-output dynamics are governed by A_m alone.

2.4.1 Algorithm

1. Build the target polynomial `A_cl_total = A_m * A0`.
2. **Dead-beat fill** (if needed): if `A_cl_total` is shorter than the Diophantine system capacity, trailing zeros are appended (= poles at z=0). This avoids a forced leading-zero in the right-hand side that would flip the sign of S and cause the closed-loop simulation to diverge. A `UserWarning` is issued; pass an explicit `A0` to control pole placement.
3. **Direct solve** (when `A_cl_total` fits the system): solve `A_eff · S_tilde + B · R = A_cl_total` directly. With no leading zeros the sign of S_tilde[0] is positive, as required by the 1/S filter.
4. **Two-step Landau fallback** (when `A_cl_total` exceeds the system): solve the reduced equation against `A_m`, then set `S = A0 · S', R = A0 · R'`.
5. Set `T = t0 · A0`, where `t0 = A_m(1) / B(1)` enforces unity DC gain.

2.4.2 Degree constraint

`deg(A_m)` alone must not exceed the system capacity `deg(A) + deg(B)` (`deg(A_eff) + deg(B) − 1` with Integrator). `deg(A0)` is unrestricted: when `A_m · A0` overflows, the two-step Landau fallback handles it automatically.

param A_cl Coefficients of the *dominant* closed-loop characteristic polynomial A_m(z) in descending powers of z. Must NOT include the observer polynomial A0 — pass only the pole locations you want to dominate the transient response.

type A_cl list

param plant_discrete_tf

Discrete plant transfer function B(z)/A(z).

type plant_discrete_tf

ct.TransferFunction

param Integrator If True, forces S to contain (z-1) to guarantee zero steady-state error for step references and exact rejection of constant disturbances (S(1) = 0). Defaults to True.

type Integrator bool, optional

param A0 Observer polynomial. Its roots are additional closed-loop poles placed faster than A_m; they

cancel from the reference path via T = t0·A0. Pass None to let the function choose poles automatically (a UserWarning is issued when dead-beat fill is applied). Defaults to None.

type A0 array-like or None, optional

returns S_tf (ct.TransferFunction): S polynomial as a discrete TF with denominator 1. R_tf (ct.TransferFunction): R polynomial as a discrete TF with denominator 1. T_tf (ct.TransferFunction): T polynomial as a discrete TF with denominator 1. H_cl (ct.TransferFunction): Closed-loop transfer function B·T / (A·S + B·R).

rtype tuple

raises ValueError If A_cl degree exceeds the controller structure capacity, or if A_m * A0 contains unstable poles.

`Utils.computeRST.`**`Compute_Desired_RST`** (*`Desired_TF`*, *`plant_discrete_tf`*, *`Integrator=True`*, *`A0=None`*)

RST synthesis from a fully specified desired closed-loop transfer function.

Computes S, R, T using the Landau observer polynomial formulation. The algorithm:

1. Solve the *reduced* Diophantine for S', R' (where `A_m = Desired_TF.den` and `A_eff = A*(z-1)` when `Integrator=True`):

```
A_eff(z) · S'(z) + B(z) · R'(z) = A_m(z)
```

2. Apply the observer polynomial factor (`T' = B_cl / B`) so that A0 cancels from the closed-loop reference-to-output path:

```
S = A0 · S',   R = A0 · R',   T = A0 · T'
H_cl = B·T / (A·S + B·R) = B_cl / A_m = Desired_TF
```

Degree constraint: `Desired_TF.den` (the dominant polynomial A_m) must satisfy `deg(A_m) <= deg(A) + deg(B)`. A0 may have any degree; it does not consume this budget.

Parameters

- **`Desired_TF`** (*`ct.TransferFunction`*) – Desired closed-loop TF B_cl(z) / A_m(z) *without* the observer polynomial. The denominator specifies dominant poles; A0 is appended separately. The numerator B_cl must be exactly divisible by B.
- **`plant_discrete_tf`** (*`ct.TransferFunction`*) – Discrete plant transfer function B(z)/A(z).
- **`Integrator`** (*`bool, optional`*) – If True, forces S to contain (z-1). Defaults to True.
- **`A0`** (*`array-like or None, optional`*) – Observer polynomial; roots are additional closed-loop poles that cancel from H_cl. Pass None for no observer poles.

Returns S_tf, R_tf, T_tf (ct.TransferFunction): Polynomial TFs with denominator 1. H_cl (ct.TransferFunction): Verified closed-loop TF ≈ Desired_TF.

Return type tuple

Raises **`ValueError`** – If degree constraint violated, A_m * A0 has unstable poles, or B_cl is not exactly divisible by B.

`Utils.utils.`**`as_csv`** (*`csv_title: str`*, *`logs`*)

Exports a SimLog to a CSV file with columns: time, output, input.

Parameters

- **`csv_title`** (*`str`*) – Output filename without the `.csv` extension.
- **`logs`** (*`SimLog`*) – Populated SimLog instance to export.

`Utils.utils.`**`Place_real_radius`** (*`plant_tf`*, *`pole_radius=0.85`*, *`steady_gain=1.0`*)

Builds a desired closed-loop transfer function with all real poles at a given radius.

Constructs a TF compatible with `Compute_Desired_RST`. The denominator degree is deg(A) + deg(B) and all poles are placed at `pole_radius`. The numerator is scaled to achieve the requested DC gain.

Parameters

- **`plant_tf`** (*`ct.TransferFunction`*) – Discrete plant transfer function.
- **`pole_radius`** (*`float, optional`*) – Desired pole radius, strictly in (0, 1). Poles closer to 1 yield a slower response. Defaults to 0.85.

- **steady_gain** (*float, optional*) – Desired closed-loop DC gain. Defaults to 1.0.

Returns Desired closed-loop transfer function with the same sampling period as `plant_tf`.

Return type ct.TransferFunction

Raises **ValueError** – If `pole_radius` is not strictly between 0 and 1.

`Utils.utils.`**poles_to_denominator** (*poles, check_stability=True, tol=0.01*)

Builds a monic denominator polynomial from discrete-time poles.

Parameters

- **poles** (*array-like*) – Desired pole locations in the z-plane.
- **check_stability** (*bool, optional*) – If True, raises `ValueError` when any pole lies on or outside the unit circle (`abs(z) >= 1`). Defaults to True.
- **tol** (*float, optional*) – Threshold below which residual imaginary parts of the computed coefficients are discarded. Defaults to 1e-2.

Returns Monic denominator coefficients in descending powers of z.

Return type np.ndarray

Raises **ValueError** – If `check_stability` is True and any pole is unstable.

2.5 Metrics & Plotting

Lightweight data logger for simulation trajectories.

`SimLog` accumulates (t, y, u) samples appended by runner functions and simulator loops. After a simulation run the three lists can be passed directly to `Metrics` or `Plotting`.

class `Metrics_Plotting.SimLog.`**SimLog**

Bases: `object`

Accumulates time, output, and input samples from a simulation run.

Each call to `log` appends one sample. After the simulation, `t_hist`, `y_hist`, and `u_hist` hold the complete trajectories as plain Python lists, ready for plotting or export.

log (*t: float, y: ndarray, u: ndarray*)

Appends one time step to the trajectory.

Both `y` and `u` are expected to be single-element arrays; `.item()` is called to store plain Python floats rather than NumPy scalars, which prevents unbounded array accumulation over long runs.

Parameters

- **t** (*float*) – Current simulation time [s].
- **y** (*np.ndarray*) – Plant output at time t, shape (1,).
- **u** (*np.ndarray*) – Control input applied at time t, shape (1,).

Performance metrics for closed-loop simulation results.

`Metrics` computes time-domain step-response metrics (rise time, settling time, overshoot) and frequency-domain stability margins (gain margin, phase margin) from either a `SimLog` or a `ct.TransferFunction`.

class `Metrics_Plotting.Metrics.`**Metrics**

Bases: `object`

Computes time-domain and frequency-domain performance metrics from simulation logs.

response_data (*Logger: SimLog, reference: float*)

Prints rise time, settling time, overshoot, and steady-state error from a closed-loop log.

Analysis is restricted to the pre-disturbance phase (t ≤ 3 s) because the disturbance applied at t = 3 s would otherwise corrupt the transient metrics.

2.5.1 Hardcoded thresholds

t ≤ 3.0 s

Pre-disturbance analysis window. The runners apply a step disturbance at t = 3 s, so any sample after this

instant reflects disturbance rejection rather than reference tracking.

10 % / 90 % of reference
Rise-time definition: the time for the output to travel from 10 % to 90 % of the reference value.

±10 % band
Settling-time tolerance band. The settling time is the last instant at which the output is still outside this band.

2.5 s to 3.0 s window
Last 0.5 s before the disturbance onset, used to estimate the steady-state output once the transient has died out.

param Logger Populated SimLog from a completed closed-loop simulation run.

type Logger SimLog

param reference Controller setpoint (target ball position).

type reference float

Stability (*TF: TransferFunction*)
Prints the gain margin and phase margin of an open-loop transfer function.
Uses `control.stability_margins` to compute both margins in one call, then converts the gain margin from a linear ratio to decibels.

Parameters **TF** (*ct.TransferFunction*) – Open-loop discrete or continuous transfer function whose stability margins are to be evaluated.

Quick-look plotting utilities for simulation results.

For publication-quality figures, use the inline plotting code in the notebooks directly; this module is intended for rapid diagnostic checks during development.

class Metrics_Plotting.Plotting.**Plotting**
Bases: object
Utility class for visualising simulation results stored in a SimLog.

plotAll (*Log, title*)
Plots the output trajectory and control input from a simulation log.
Opens two separate Matplotlib figures: plant output y(t) and control input u(t), both as functions of time.

Parameters
- **Log** (*SimLog*) – Populated SimLog instance containing t_hist, y_hist, u_hist.
- **title** (*str*) – Title string applied to both figures.

3 Indices

- genindex
- modindex